Technical Report: AI-Driven Development Ecosystem for "The Impostor Game"

Course: AI Models Pre-training and Training in Digital Reality Environments and 3D/XR Simulators Building

Authors: Andrés Rey Luna & Javier García Palomares

Project Scope: Real-Time Multiplayer Entertainment Web Application

1. Project Overview & Advisory Role

In this project, we successfully designed and developed *The Impostor Game*, a real-time multiplayer hidden-screen party application. To overcome architectural uncertainties, streamline implementation bottlenecks, and maximize our agile development workflow, we integrated a collaborative Generative AI pipeline. Throughout this engineering lifecycle, we utilized **Google Gemini Ultra V4** as our Lead AI Assistant and Strategic Advisor.

Gemini guided our high-level decision-making, systematically recommended specific specialized software intelligence models for each development sub-stage, and provided the technical frameworks required to build a lightweight, responsive multiplayer system without physical or computational infrastructure overhead.

2. Phase 1: SPA Architecture Benchmarking & Tool Selection

A critical phase of the project was determining the optimal technical infrastructure to ensure instant, friction-free access without requiring users to download heavy native software or undergo complex installation loops. Following a thorough methodology of benchmarking and documenting our initial architectural exploration, we consulted our Lead AI Assistant for a comprehensive market analysis of the best generative AI tools specialized in creating Single Page Applications (`SPA`).

- **The Iterative Evaluation:** We spent an initial phase testing, experimenting, and benchmarking several state-of-the-art open-source and proprietary coding models (including *DeepSeek-Coder* and *Meta Llama 3 Code*). During these structural trials, we encountered significant hurdles regarding script coherence, as early generative outputs failed to properly structure synchronous callback loops and handle clean, cross-device DOM manipulations simultaneously.
- **The Breakthrough:** After rigorous trial, error, and metric-based analysis, we found a specialized tool that truly convinced us and fit our exact production criteria: **v0 by Vercel**. This specialized network provided the high level of satisfaction we were looking for, successfully generating an optimized Single Page Application web architecture. It accurately mapped out our `Node.js` and `Express` structural backbone while ensuring the frontend remained entirely framework-free, lightweight, and performant.

3. Phase 2: Technical Hurdles & Prompt Engineering for WebSockets

Once the baseline SPA architecture was firmly established, the implementation of the synchronized server-client game state became a rigorous process of prompt refinement and behavioral constraints. We combined our selected SPA environment with targeted sessions in **GitHub Copilot** to open permanent, bidirectional communication channels via `Socket.io`.

> **ATTEMPT 1: STANDARD HTTP POLLING LOOP (FAILED IMPLEMENTATION)**
>
> **The Problem:** Early automated scripts relied heavily on standard HTTP asynchronous pooling requests. During intense local network simulation with multiple active nodes, this approach caused severe performance overhead, race conditions, and unacceptable delays during real-time voting updates.

The Optimization Attempt (Prompt Refinement): We adapted our prompt inputs dynamically when the AI instances failed to address local network congestion. We explicitly instructed the system via targeted constraints to drop heavy external code dependencies and transition the core voting mechanics into an event-driven paradigm using pure **Vanilla JavaScript (ES6+)**.

This strict prompt constraint guaranteed that game state updates, screen transitions, and hidden-word assignments occurred simultaneously, atomically, and instantly across all connected mobile viewports without requiring an active server database storage layer.

4. Phase 3: UX/UI Optimization and Console Interface Deployment

To resolve the tedious and error-prone process of requiring players to enter manual IP addresses and local ports on mobile browsers, we pivoted from structural backend models to a specialized optimization session with **Claude 4.5 Sonnet**.

- **The Innovation:** This model suggested a brilliant, frictionless approach to maximize user experience (UX): utilizing the host's terminal console as a direct visual entry point.
- **Multimodal Deployment:** Claude successfully generated deployment scripts combining the `internal-ip` module for dynamic LAN resolution and the `qrcode-terminal` module to render connection matrices instantly in the server console. Consequently, invited players simply needed to point their smartphone cameras at the host's screen to parse the QR code and instantly join the active game lobby.

5. Methodology & AI Concepts Applied

The successful delivery of the system relied on three fundamental AI methodologies:

- **AI-Human Collaboration:** We implemented a hierarchical "Co-pilot" workflow where Google Gemini provided the high-level strategic roadmap, our benchmarked SPA tool (v0) established the core application files, and specialized localized instances executed granular syntax debugging.
- **Prompt Refinement & Iteration:** We demonstrated technical adaptability by modifying language constraints and setting strict runtime boundaries when generative instances outputted conflicting event listeners. This achieved zero resource overhead and absolute database-free data persistence.
- **Multimodality Simulation:** We utilized advanced prompt frameworks to ensure the AI understood the direct relationship between our responsive CSS3 layout constraints and the real-time state machine governing the game flow.

6. Conclusion & AI-Mapped Milestones

The development of *The Impostor Game* successfully achieved its educational and technical goals by blending modern web technologies with an event-driven generative architecture. Managing the script's expansion and documenting our technical "failures" gave us deep insight into the current state of Generative AI software creation.

> ***Future Roadmap:*** *Looking ahead, we have leveraged our Lead AI Assistant to map out our upcoming cloud deployment milestones. The next steps of development will utilize cloud platform automation tools to transition our architecture from local LAN to public WAN domains, implementing an automated Room Code system to securely connect real-time gaming sessions seamlessly between Poland and Spain.*